SIMATS ENGINEERING

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA16 Data Warehousing and Data Mining

Assignment 2 – CO2

CO2 Coverage: *Data mining from a database perspective, data mining functionalities, steps in the data mining process (KDD), architecture and classification of data mining systems, basic data mining tasks and issues, data dispersion, data cleaning, integration, transformation, reduction and discretization.*

Course Outcome: CO2 – Analyse data dispersion techniques and list the various algorithms available for mining the data.

Nature: Individual | **Duration:** 1 Week | **Total Marks:** 50

Q1. Matching Business Problems to Mining Functionalities (5 marks)

The six TeleNova problems are mapped below. "Supervised" means the target label is known in the training data. Algorithm names are the standard ones from the CO3/CO4 syllabus.

Problem	Functionality	Learning type and pattern produced	Data attributes and source	Candidate algorithms and reason
P1 Plans and add-ons bought together	Association analysis	Unsupervised. Rules such as "plan A → add-on B" with support, confidence and lift.	Purchase transactions: customer ID, plan ID, add-on codes, purchase date. Source: transactional billing/order database, summarised in a sales fact table in the warehouse.	**Apriori**: level-wise candidate generation; easy to explain to marketing. **FP-Growth**: no candidate generation; needed for 8 M customers and a large add-on catalogue.
P2 Will a customer leave next month?	Classification (prediction of a categorical label)	Supervised. A model that outputs churn / no-churn, or a churn probability.	Labelled history: monthly voice minutes, data GB, SMS, tenure, plan, complaints, payment delays, and a churn flag from the following month. Source: warehouse (historical customer snapshots).	**Decision-tree induction (ID3/C4.5/CART)**: produces readable rules the retention team can act on. **Naïve Bayes**: very fast, scales to 8 M rows, a strong baseline.
P3 Natural usage segments, no labels	Clustering	Unsupervised. Groups with similar usage profiles (for example heavy-data, night-caller, low-usage).	Normalised per-subscriber features: average monthly minutes, data GB, SMS count, night share, roaming days, recharge	**k-means**: fast and simple; k chosen by elbow or silhouette. **BIRCH**: built for very large data (one scan, CF-tree), suited to 8 M rows.

			frequency. Source: warehouse usage-summary table.	
P4 SIM cards with very different calling behaviour	Outlier analysis	Unsupervised. An outlier score or flag per SIM (possible fraud, SIM-box or bot activity).	Per-SIM aggregates: calls per day, mean duration, distinct numbers called, international share, night-time calls. Source: call-detail-record (CDR) aggregates in the warehouse.	**DBSCAN**: points in sparse regions are labelled noise. **LOF (local outlier factor)**: compares each SIM with its local density, so it works when behaviour groups differ in density.
P5 Characteristics of high- vs low-value customers	Concept description (characterisation and comparison)	Descriptive, class-based. A class label (high/low value) is given, but the output is a generalised profile and a contrast, not a predictive model.	Customer dimension and revenue fact: ARPU, tenure, plan tier, data use, age band, region, payment mode, value class. Source: data warehouse (dimension and fact tables).	**Attribute-oriented induction (AOI)**: generalises attributes into readable class profiles and contrasts them. **Decision tree in descriptive mode**: shows which attributes separate the two classes (used only to describe, not to predict).
P6 How monthly data usage changed over 24 months	Evolution analysis (time series / sequential patterns)	Unsupervised over time. Trends, seasonality, and shared patterns such as "usage rises for six months, then falls".	Monthly data volume per subscriber (24 time-stamped points), plus plan and cohort. Source: warehouse time-variant monthly fact table.	**Sequential pattern mining (GSP / PrefixSpan)** on trend symbols (up / flat / down). **k-means on normalised 24-month curves** to group customers with similar trend shapes.

Evaluation of the marketing manager's proposal: one all-purpose algorithm

The proposal should be rejected as stated. The analysis above shows six different learning tasks, and no single algorithm fits all of them:

- **Different outputs.** P1 needs rules, P2 a classifier, P3 clusters, P4 outlier scores, P5 class profiles, and P6 time-ordered patterns. An algorithm built for one output form does not give the others.
- **Different learning settings.** P2 and P5 use labels; P1, P3, P4 and P6 do not. k-means cannot predict churn, and a decision tree cannot find outliers without labelled examples.
- **Different data types.** P1 works on sets of items, P2 to P4 on numeric features, and P6 on time series. Each needs its own representation.

- **Different evaluation measures.** Confidence and lift (P1), recall and AUC (P2), silhouette and cluster stability (P3), precision on confirmed fraud (P4), and trend validity (P6). A single algorithm cannot be judged by one score.
- **Scale differs.** Apriori becomes expensive on very large item catalogues, while k-means and Naïve Bayes scale well. FP-Growth or BIRCH may be needed for 8 M subscribers.
- **No free lunch.** No algorithm is best across all tasks; this is a basic result in learning theory.

What is valid in the proposal: a shared platform (for example Weka, KNIME or Python scikit-learn) reading from one warehouse, with common preprocessing, is sensible and cheaper to run. The recommendation is therefore to buy or build one **mining platform** that offers a toolkit of functionality-specific algorithms, not one all-purpose algorithm. Budget should also go into data quality, because all six problems depend on it.

Q2. The KDD Process, System Architecture and Mining Issues (5 marks)

2.1 KDD steps mapped to MediCore activities

KDD step	Activities MediCore must perform
Selection	Define the cohort (index admissions in the study period). Define the outcome: readmission within 30 days, separating planned from unplanned readmissions and excluding in-hospital deaths. Select the tables from the admission, laboratory, pharmacy and billing systems. Select attributes: age, diagnosis codes, comorbidities, length of stay, discharge disposition, last lab values, discharge medicines. Obtain data-use approval.
Preprocessing (cleaning and integration)	Link the four systems through a master patient ID (entity resolution). Handle missing lab values and impossible values (for example negative length of stay). Remove duplicate admission records. Map ICD-9 and ICD-10 codes to one version. Reconcile billing and discharge dates that disagree.
Transformation	Aggregate labs per admission (last value, maximum, count of abnormal results, trend). Derive features: prior admissions in the last 180 days, comorbidity index, count of discharge drugs (polypharmacy). Normalise continuous variables, encode categories, and create age bands. Align time windows to the discharge date.
Mining	Classification of 30-day readmission risk (logistic regression, decision tree, or Naïve Bayes). Association rules over discharge diagnoses and medicines to find combinations linked to readmission.
Interpretation and evaluation	Filter patterns by support, confidence, lift, AUC and recall. Have clinicians judge whether patterns are plausible and actionable. Validate on a later time period (temporal validation). Check performance across age, sex and other groups for bias. Present results to clinical governance.

2.2 Architecture of a data mining system for MediCore

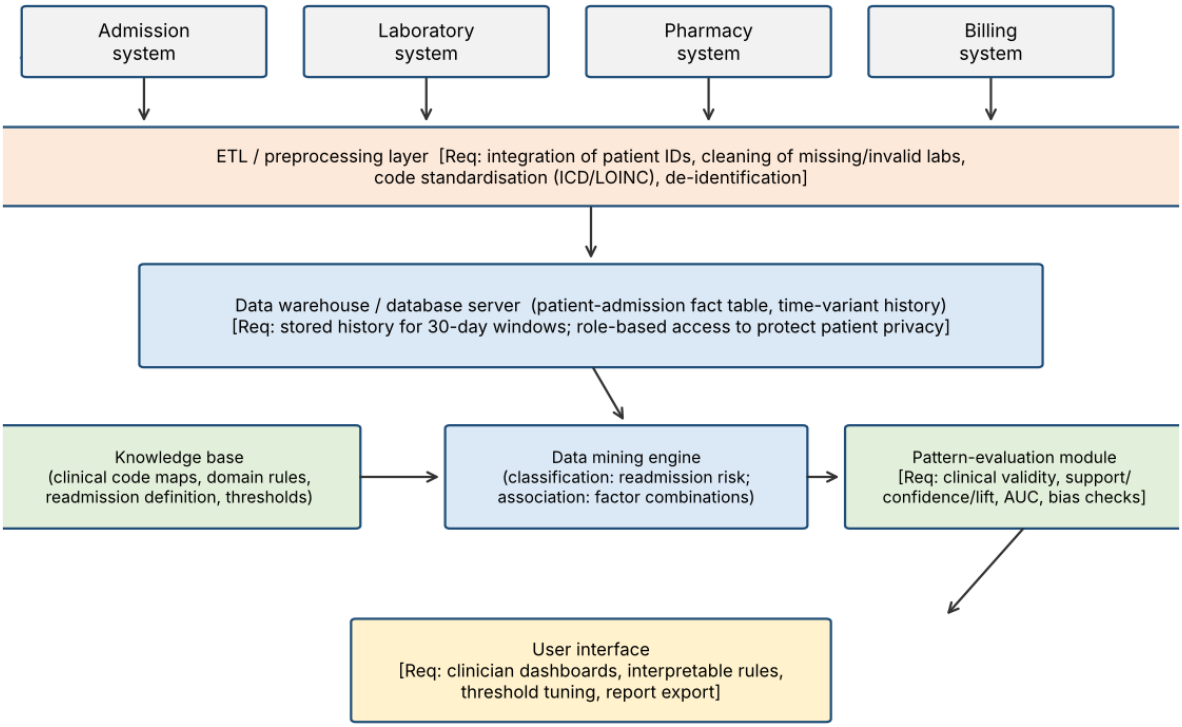


Figure 1. Architecture of the MediCore mining system (shaded boxes mark where the requirements are handled).

Requirement	**Where it is handled**
Integrate four source systems	ETL / preprocessing layer (master patient ID, schema mapping)
Clean missing and invalid lab values	ETL layer (cleaning rules) with domain rules held in the knowledge base
Consistent clinical coding	Knowledge base (ICD and LOINC mapping tables)
Patient history for 30-day windows	Data warehouse (time-variant patient-admission fact table)
Privacy and access control	Database / warehouse server (role-based access, de-identified views, audit log)
Classification and rule mining	Data mining engine
Interestingness and clinical validity of patterns	Pattern-evaluation module (support, confidence, lift, AUC, bias checks)
Clinician use	User interface (dashboards, rule display, threshold tuning, export)

2.3 Classification of the system on four views

View	**Classification for MediCore**
Kinds of data	Relational, multi-source electronic health records (structured tables);

	temporal data (lab results over time); optionally unstructured clinical notes in a later phase.
Kinds of knowledge	Predictive knowledge (a readmission-risk score for each new discharge) and descriptive knowledge (association rules showing risk-factor combinations).
Techniques used	Database and OLAP techniques over the warehouse; statistical methods (logistic regression); machine-learning methods (decision trees, Naïve Bayes); association-rule mining (Apriori).
Applications	Healthcare: clinical decision support, discharge planning and hospital quality improvement.

2.4 Data mining issues and mitigations

Issue (category)	**Why it matters at MediCore**	**Mitigation**
Methodology: outcome definition and confounding	Planned readmissions are not preventable failures. Age and illness severity confound the effect of any single factor.	Define the outcome with clinicians. Adjust for confounders. Model planned and unplanned readmissions separately. Validate on a held-out period.
User interaction	Clinicians need explainable, actionable output, not model internals.	Prefer rule-based or tree-based outputs. Build an iterative loop with clinicians. Provide dashboards with threshold tuning.
Efficiency and scalability	Several years of records across four large systems, including lab tables.	Incremental loading. Pre-aggregate per admission (data cube). Develop on a sample, then run the full set. Index the warehouse; use parallel processing.
Diversity of data types	Codes, numeric labs, dates, and free-text notes; coding systems differ between systems.	Standardise codes with mapping tables (ICD, LOINC). Engineer features over fixed time windows. Add NLP for notes in a later phase.
Privacy and social or ethical issues	Identifiable health data. Risk of discriminatory risk scores.	Pseudonymise patient IDs. Use role-based access and audit logs. Comply with applicable health-data law (for example HIPAA in the US or the DPDP Act 2023 in India). Run fairness checks across groups and obtain ethics approval.
Data quality: missing labs	A test not ordered is not the same as a normal result. Treating missing as normal creates false patterns.	Add missingness indicators as features. Use clinical domain rules when filling values. Report sensitivity to the imputation method.

2.5 Reasoned answer to the director

The director is partly right and partly wrong.

- **A SQL query** answers a question the analyst already has. For example, "How many patients were readmitted within 30 days in 2025?" returns a known fact. It does not reveal which combinations of conditions matter.
- **A statistical summary** describes data or tests one hypothesis at a time. For example, "The 30-day readmission rate is 14% overall and differs by ward." It can compare groups, but it cannot search a few hundred attributes for interactions.
- **A mining task** asks an unknown question: which combinations of attributes predict readmission, and how accurately can we predict it for a new patient at discharge? An illustrative (hypothetical) discovered rule could be: "patients with many discharge medicines, a prior admission in the last 180 days, and a comorbidity score above a threshold have a much higher readmission rate than average." No one would think to write that query. The result is also a predictive model, applied to new admissions and validated on unseen data.

Conclusion: SQL and statistics remain essential for defining the cohort, cleaning the data and checking results, so the director is right that they are needed. Mining adds value when the task is to discover unknown interactions and to predict. Statistics and mining overlap (logistic regression is both), so the right answer is to use both.

Q3. Describing and Comparing Data Dispersion (7 marks)

Data (n = 24, in minutes, already sorted): 28, 31, 33, 35, 35, 36, 38, 40, 41, 42, 42, 43, 45, 46, 48, 50, 52, 53, 55, 58, 61, 64, 72, 95. The sum is 1143.

3.1 Central tendency

Measure	Working	Result
Mean	1143 / 24	**47.63**
Median	n is even: average of the 12th and 13th values = (43 + 45) / 2	**44**
Mode	35 and 42 each occur twice (bimodal; no single mode)	**35 and 42**
Midrange	(28 + 95) / 2	**61.5**

Relationship between mean and median. The mean (47.6) is larger than the median (44). Values such as 95, 72 and 64 pull the mean up, so the distribution is **right-skewed** (positively skewed). The median is the better guide to a typical order here.

3.2 Spread

Quartile convention used: Tukey's median-of-halves method. Each half has 12 values, so Q1 is the median of the lower 12 values and Q3 is the median of the upper 12 values.

Measure	Working	Result
Range	95 − 28	**67**
Q1	Median of {28, 31, 33, 35, 35, 36, 38, 40, 41, 42,	**37**

	42, 43} = (36 + 38) / 2	
Q3	Median of {45, 46, 48, 50, 52, 53, 55, 58, 61, 64, 72, 95} = (53 + 55) / 2	**54**
IQR	Q3 − Q1 = 54 − 37	**17**
Sum of squared deviations	$\Sigma(x - 47.625)^2$ (computed value 5119.63)	5119.63
Sample variance s^2	5119.63 / (24 − 1)	**222.59**
Sample standard deviation s	√222.59	**14.92**
Population variance σ^2 (for reference)	5119.63 / 24	213.32 (σ = 14.61)

I use sample variance (divide by n − 1) because the 24 orders are a sample of deliveries. Excel's VAR.S and STDEV.S give the same values; VAR.P and STDEV.P give the population values.

3.3 Five-number summary, boxplot and outliers

Five-number summary: **Min 28, Q1 37, Median 44, Q3 54, Max 95.**

Outlier fences (1.5 × IQR rule): lower fence = 37 − 1.5 × 17 = 11.5; upper fence = 54 + 1.5 × 17 = 79.5. The upper whisker ends at 72, the largest value inside the fence. **95 lies beyond 79.5, so it is an outlier.** No value is below 11.5.

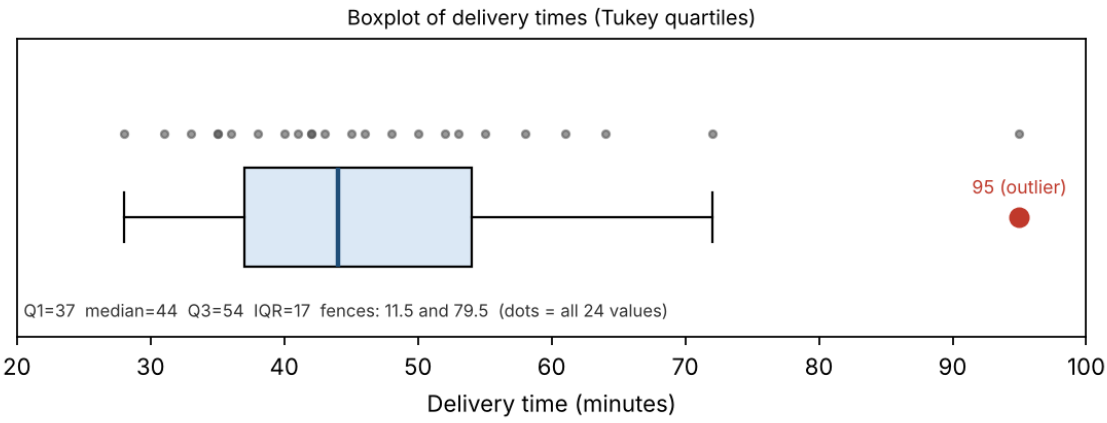


Figure 2. Boxplot of the 24 delivery times with the outlier at 95 marked.

Note on the quartile convention: Excel's QUARTILE.INC (linear interpolation) gives Q1 = 37.5 and Q3 = 53.5, so IQR = 16 and the fences become 13.5 and 77.5. 95 is still the only outlier. The conclusion does not depend on the convention, but the figures do.

3.4 Remove the largest value (95) and recompute

Measure	With 95 (n = 24)	Without 95 (n = 23)	Change	% change
Mean	47.63	45.57 (1048 / 23)	−2.06	−4.3%
Median	44	43	−1	−2.3%
Mode	35, 42	35, 42	none	0%
Midrange	61.5	50.0	−11.5	−18.7%

Range	67	44	−23	−34.3%
Q1 / Q3	37 / 54	36 / 53	−1 / −1	small
IQR	17	17	none	0%
Sample variance	222.59	126.26	−96.33	−43.3%
Sample std deviation	14.92	11.24	−3.68	−24.7%

Which measures changed most, and why? Variance (−43%), range (−34%), midrange (−19%) and standard deviation (−25%) changed most. Each depends on the extreme value: squared deviations amplify it for variance and SD, and the range and midrange are built directly from the maximum. The mean moved moderately, because 95 is one value among 24. The median, IQR and mode barely moved, because they depend only on the middle of the ordered data. This is why robust measures are preferred when outliers are present.

3.5 Classification of the measures and the warehouse setting

Measure	Class	Why
Mean	Algebraic	Computed from sum and count, which are each distributive.
Median	Holistic	Needs the whole ordered data; a partial median cannot be merged exactly from partition medians.
Range	Algebraic	Computed from max and min, each of which is distributive.
Variance / standard deviation	Algebraic	Computed from count, sum and sum of squares, which are distributive.
Midrange (for reference)	Algebraic	Computed from max and min.

Meaning for warehouse partitions. A **distributive** measure can be computed for each partition and combined directly (sum and count, for example). An **algebraic** measure can be computed exactly from a fixed set of distributive values per partition: each partition stores (count, sum) for the mean, (count, sum, sum of squares) for variance, and (max, min) for range. Those small summaries are merged, and the exact result is obtained without rereading the data. A **holistic** measure such as the median cannot be merged exactly; the system must either hold all values or use an approximation (for example a histogram or quantile sketch), which may be slightly off. None of the five asked measures is distributive on its own; the mean, variance and range are algebraic, and the median is holistic.

3.6 Verification with a tool

I verified the figures with Python (numpy and statistics). Results agreed for the mean (47.625), median (44), midrange (61.5), range (67), sample variance (222.59), sample SD (14.92), and the outlier at 95. The only differences come from conventions: numpy's default percentile gives Q1 = 37.5 and Q3 = 53.5 (interpolation, as in Excel's QUARTILE.INC), and population versus sample variance (213.32 versus 222.59). Each difference is explained above. The verification script is q3_q6_verification.py.

3.7 Recommendation to operations

Recommended measure for a "typical" delivery time: the median, 44 minutes. It is the value that half of orders beat and half exceed. It is not pulled by the 95-minute order, so it describes typical service accurately.

Report in addition:

- **The IQR and quartiles (37 to 54 minutes):** they show the middle 50% of orders and how consistent service is. A 17-minute spread is a fair description of routine variation.
- **A high percentile, for example the 90th percentile (about 63 minutes by linear interpolation):** if the company promises a time, promise one that most customers receive. Four of 24 orders (17%) took more than 60 minutes, so promising 44 minutes would be broken for about half of customers.
- **The outlier(s) listed separately (95 minutes):** a single extreme order can be investigated for root cause (dark-store stock-out, rider delay) without distorting the headline figure.
- **The mean (47.6 minutes), for cost and capacity planning**, since totals depend on the mean rather than the median.

Q4. Cleaning Noisy and Incomplete Data (6 marks)

4.1 Defects in the extract

Record	Field and value	Defect	Class
M102	kWh = (missing)	Missing measurement	Missing value
M105	Household type = (missing)	Missing categorical attribute	Missing value
M104	kWh = −14	Negative energy consumption; not physically meaningful for a household meter (unless it is solar export, which must be confirmed)	Invalid value / noise
M105	Voltage = 2300 V	About ten times the nominal 230 V; probably a decimal-place or unit error	Noise / invalid value
M103	Region = "north"	Same category written in a different case from "North"	Inconsistency
M106	Region = "S."	Abbreviation of "South" instead of the standard label	Inconsistency
M103	kWh = 38	Not a defect: it equals the largest of the 12 other readings. Flag it for verification only	(none)

4.2 Handling strategy for each defect

Defect	Strategy chosen	Justification
M102 kWh missing (type = Apartment)	**Class-wise mean** (Apartment)	Consumption depends strongly on household type (houses about 38 kWh, apartments 5 to 11 kWh in this extract). The global mean (21.1) would wrongly inflate an apartment's value. Known apartment

		readings are 5 and 11, giving 8.0 kWh. Only two values support this, so it is a fallback; regression on voltage, region and time is preferred when more data exist.
M105 household type missing	**Fill manually** from the meter registry; if it cannot be found, use the **most probable value**	A categorical value should be looked up, not guessed. Apartments are the most frequent type among known records (3 of 5), so this is the fallback if the registry is unavailable.
M104 kWh = −14 (invalid)	**Ignore the tuple** for kWh-based mining (or set to missing and do not impute)	The only valid house reading is 38 kWh, too few to estimate a reliable class-wise mean. Imputing from one value would create a false pattern. Keep the record in a flagged table and check whether it is solar export.
M105 voltage = 2300	**Correct** to 230.0 if the meter log confirms a decimal error; otherwise **fill with the attribute mean**	Valid voltages are 230, 231, 229, 232 and 230, with mean 230.4. Voltage is almost constant, so the mean is harmless. Leaving 2300 in would give a mean of about 575 V, which is clearly wrong.
Region "north" and "S."	**Transformation by mapping table**: north → North; S. → South	Not missing or noisy; the same category is spelled differently. A mapping table is the correct fix and avoids treating them as separate categories.

4.3 Equal-frequency binning (depth 4) of the 12 sorted readings

Values: 5, 9, 11, 14, 18, 19, 23, 26, 27, 30, 33, 38. Three bins of four values each.

Bin	Values	Bin mean	Smoothed by bin means	Smoothed by bin boundaries
1	5, 9, 11, 14	9.75	9.75, 9.75, 9.75, 9.75	5, 5, 14, 14
2	18, 19, 23, 26	21.50	21.5, 21.5, 21.5, 21.5	18, 18, 26, 26
3	27, 30, 33, 38	32.00	32, 32, 32, 32	27, 27, 38, 38

Working for boundary smoothing: each value moves to the nearer of its bin's minimum and maximum. In bin 1, 9 is 4 from 5 and 5 from 14, so it becomes 5; 11 is 6 from 5 and 3 from 14, so it becomes 14. Bins 2 and 3 follow the same rule. Bin-mean smoothing removes within-bin variation completely; boundary smoothing keeps the bin extremes and so preserves more of the range.

4.4 Regression and clustering for noise, and why 2300 V is treated differently from 5 kWh

- **Regression:** fit kWh as a function of household type, voltage, region and hour. Readings with residuals far beyond the normal range (for example more than three standard deviations) are flagged as noise, and the fitted value replaces them. This smooths noise while keeping real variation.

- **Clustering:** cluster readings on kWh, voltage and household type (for example k-means). Points far from every centroid, or in very small clusters, are noise candidates and can be replaced with the centroid or a regression prediction.
- **2300 V versus 5 kWh:** 2300 V is outside the physical range for a 230 V residential supply (the nominal ±10% band is about 207 to 253 V). It is a known error type, so a **domain rule** should catch it and correct or remove it before any statistics are applied. The value of 5 kWh is valid, low-consumption behaviour, and it is a legitimate observation that should be kept. A statistical outlier test might flag it because it is low, but that flag should not cause its removal. Rules first, statistics second.

4.5 Inquiry: the effect of mean imputation on dispersion and mining

Take the 12 readings above. Their sample variance is 103.72 (standard deviation 10.18). If one missing kWh is filled with the global mean of 21.08, the 13 values have sample variance 95.08 (standard deviation 9.75). The variance has fallen by about **8%**, and this is from a single filled value.

- **Dispersion is understated.** Filled values sit at the centre, so the spread shrinks, and variance and standard deviation are underestimated. Any later confidence interval or threshold based on spread becomes too narrow.
- **Relationships are weakened.** The mean is the same for apartments and houses, so the fill destroys the difference between household types. Correlations with other attributes move towards zero, and regression coefficients are biased towards zero.
- **False patterns appear in clustering and outlier analysis.** Many filled records form a dense cluster at the mean. The algorithm then finds a "typical" group that exists only because of the fill, and real outliers near the centre are hidden.
- **Bias if missingness is not random.** If high-consumption meters are more likely to fail to report, the mean fill is systematically wrong.

When to avoid it: when the attribute is strongly related to other attributes (such as household type here), when the missing fraction is large, when the data are missing not at random, and when the attribute is central to clustering or classification. **Better alternatives:** class-wise mean, regression or model-based imputation, a missingness indicator added as a feature, and sensitivity analysis comparing results under different fills.

Q5. Integration and Transformation of Data (7 marks)

5.1 Part A: integration conflicts and their resolution

Conflict type	Where it appears	Resolution in the integrated schema
Entity identification	student_id (Admissions) and sid (LMS) are keys for the same student. The LMS has no date of birth, so matching relies on the ID mapping and name similarity.	Use **student_id** as the canonical key. Build a mapping table sid → student_id, validated by name agreement, and send unmatched records to manual review.
Naming (same	name / full_name; scholarship /	Rename to one schema: student_id,

attribute, different names)	scholarship_flag; student_id / sid	name, dob, hostel_status, scholarship, attendance_pct, marks.
Unit and format	Date of birth stored as text in unknown order (DD/MM or MM/DD); attendance held as a percentage in the LMS; name order may differ ("First Last" versus "Last, First").	Store dates in ISO format (YYYY-MM-DD). Keep attendance as a percentage 0 to 100. Store name as "First Last" after parsing.
Value conflict	A student may be recorded as having a scholarship in one system and not in the other. Hostel status exists only in Admissions.	Name the authoritative source for each attribute (for example the scholarship office for scholarship status). Keep the source in a provenance column. Flag disagreements for review.
Redundancy	scholarship and scholarship_flag hold the same information; name and full_name too. Attendance and marks are strongly related (Part B).	Keep one attribute per concept. Detect remaining redundancy with correlation (numeric) and chi-square (categorical) analysis.

Why integration creates redundancy: the same concept is stored under different names, some attributes can be derived from others (age from date of birth), and some attributes are naturally correlated. **Correlation analysis** detects the third kind. Pearson's r flags numeric pairs with |r| close to 1, and the chi-square test flags categorical pairs that are not independent. Either result tells the designer whether to keep both attributes.

5.2 Part B(i): Pearson correlation of attendance and marks

Attendance x = 60, 70, 75, 80, 85, 90 (mean 76.67). Marks y = 52, 58, 66, 70, 79, 85 (mean 68.33).

x	**y**	**$x - \bar{x}$**	**$y - \bar{y}$**	**$(x - \bar{x})(y - \bar{y})$**	**$(x - \bar{x})^2$**	**$(y - \bar{y})^2$**
60	52	−16.67	−16.33	272.2	277.8	266.8
70	58	−6.67	−10.33	68.9	44.4	106.8
75	66	−1.67	−2.33	3.9	2.8	5.4
80	70	3.33	1.67	5.6	11.1	2.8
85	79	8.33	10.67	88.9	69.4	113.8
90	85	13.33	16.67	222.2	177.8	277.8
Sum				**661.7**	**583.3**	**773.3**

$r = 661.7 / \sqrt{(583.3 \times 773.3)} = 661.7 / \sqrt{451{,}111} = 661.7 / 671.6 =$ **0.985**.

Interpretation: a very strong positive linear relationship. Students with higher attendance tend to score higher. $r^2 = 0.97$, so about 97% of the variation in marks is shared with attendance. This is **redundancy**: the two attributes carry almost the same information for many models. Keep one (marks is usually the target) or both only if a model needs them and multicollinearity is controlled. Correlation does not show causation.

5.3 Part B(ii): chi-square test of hostel status and scholarship

H_0: hostel status and scholarship are independent. H_1: they are dependent. Significance level 0.05; critical value for 1 degree of freedom = 3.84.

Expected counts E = (row total × column total) / grand total (N = 700):

Cell	Observed O	Expected E	$(O - E)^2 / E$
Hostel, scholarship	120	300 × 200 / 700 = 85.71	$34.29^2 / 85.71 = 13.71$
Hostel, no scholarship	180	300 × 500 / 700 = 214.29	$34.29^2 / 214.29 = 5.49$
Day scholar, scholarship	80	400 × 200 / 700 = 114.29	$34.29^2 / 114.29 = 10.29$
Day scholar, no scholarship	320	400 × 500 / 700 = 285.71	$34.29^2 / 285.71 = 4.11$
Total			**$\chi^2 = 33.6$**

Degrees of freedom = $(2 - 1)(2 - 1) = 1$. Since $33.6 > 3.84$ ($p \approx 0.000000007$), **reject H_0**: hostel status and scholarship are **dependent** at the 0.05 level. Every expected count exceeds 5, so the test is valid. The observed proportions show the direction: 40% of hostel students hold a scholarship (120 of 300) against 20% of day scholars (80 of 400). With Yates' continuity correction the statistic is 32.6, which still rejects H_0. Dependence is an association, not a causal effect.

5.4 Part C(i): three normalisation methods on family income

Values: 20,000; 35,000; 50,000; 80,000; 150,000. Mean = 67,000; population standard deviation = 46,000; min = 20,000; max = 150,000.

Income (₹)	Min-max (0 to 1): (v − 20,000) / 130,000	Z-score: (v − 67,000) / 46,000	Decimal scaling: v / 10^6 (j = 6)
20,000	0.000	−1.022	0.020
35,000	0.115	−0.696	0.035
50,000	0.231	−0.370	0.050
80,000	0.462	0.283	0.080
150,000	1.000	1.804	0.150

Min-max keeps the shape and maps to [0, 1], but a single extreme value compresses the rest. Z-score centres the data and expresses each value in standard deviations; with the sample standard deviation (51,430) the values become −0.914, −0.622, −0.331, 0.253, 1.614. Decimal scaling uses the smallest power of ten (10^6) that makes every absolute value below 1.

5.5 Part C(ii): Euclidean distance before and after normalisation

Student 1: (age 19, income 35,000). Student 2: (age 21, income 80,000). Assumed ranges: age 17 to 25 (range 8); income 20,000 to 150,000 (range 130,000).

- **Before:** $d = \sqrt{((19 - 21)^2 + (35{,}000 - 80{,}000)^2)} = \sqrt{(4 + 2{,}025{,}000{,}000)} \approx$ **45,000.0**. Age contributes 4 out of 2.025 billion, so the distance is entirely income.

- **After min-max:** age 19 → 0.25, age 21 → 0.50; income 35,000 → 0.115, income 80,000 → 0.462. d = $\sqrt{((0.50 - 0.25)^2 + (0.462 - 0.115)^2)} = \sqrt{(0.0625 + 0.1198)} = \sqrt{0.1823} \approx$ **0.427**.

What it reveals: without normalisation, the attribute measured in the largest units (income) decides all distance-based results. Age, which may matter, is ignored. After normalisation both attributes contribute on a comparable scale (0.0625 for age and 0.1198 for income). Distance-based methods (k-NN, k-means, DBSCAN, LOF) therefore need normalisation, or attribute weights, before mining. Normalised distances are unitless and are not comparable with the original units.

5.6 Part C(iii): aggregation, generalisation and attribute construction

Operation	Example for this data
Aggregation	Average marks per hostel status: the mean of marks for hostel students compared with day scholars, summarised from individual records.
Generalisation	Replace income values with bands: 20,000 and 35,000 become "low income"; 80,000 becomes "middle income"; 150,000 becomes "high income". Age 19 becomes "young adult".
Attribute construction	Create a new attribute scholarship_eligible = 1 if income < 50,000, otherwise 0, or an attendance gap = 100 − attendance_pct, which may be useful for predicting low marks.

Q6. Data Reduction and Discretization (7 marks)

6.1 Part A: choosing reduction strategies for 200 sensors and 5 million rows per month

Strategy	Technique	When it suits this data
Data-cube aggregation	Summarise raw readings to hourly or per-shift means, minima and maxima.	When failure patterns show at coarser time granularity. It reduces rows greatly while keeping summary statistics. It may hide short spikes, so keep the raw data for the failure windows.
Attribute subset selection	**Forward selection**: start empty, add the best sensor each step. **Backward elimination**: start with all 200, remove the weakest. **Decision-tree induction**: keep the sensors the tree uses for splits.	Useful because many sensors are redundant or irrelevant. Decision-tree induction is preferred here because it is labelled (failure) data and it ranks sensors by how well they separate failures. Forward selection is cheaper when few sensors matter; backward elimination needs more compute.
Dimensionality reduction	**PCA** (principal component analysis) on standardised sensors.	Suitable when many sensors are correlated (for example several vibration channels on one motor). It loses direct meaning, so it is less suitable if engineers must explain the result to a sensor.
Numerosity reduction	**Sampling**, **histograms**, and **clustering** (replace a cluster by its	Sampling is most useful because of the very large row count (see 6.2).

	centroid and size).	Histograms suit distribution summaries. Clustering suits compressing repeated normal operating states.
Compression	Wavelet transform or lossy encoding of the signals.	Helpful for storage of raw high-frequency signals, but the mining algorithm must work on the decompressed or transformed data. It does not reduce the mining cost by itself, so it is secondary here.

6.2 Sampling strategies compared (failures are 0.5% of rows)

Strategy	**How it works**	**Behaviour with 0.5% failures**
Simple random sampling without replacement (SRSWOR)	Each row can be chosen at most once.	Preserves the 0.5% rate on average, but the number of failures in a small sample is random. At 1% (50,000 rows) only about 250 failures are expected; a 0.2% sample might have very few.
Simple random sampling with replacement (SRSWR)	Rows can be chosen more than once.	Duplicates add no information and can inflate the apparent size of the failure class. No advantage here.
Cluster sampling	Select whole groups (for example machine-days) at random.	Risky: if failures cluster in a few machines or days, a sample can contain none, or many from one event. Good for cost, poor for rare-event coverage.
Stratified sampling (by failure label)	Split into the failure and normal strata and sample each.	**Recommended.** Keep all 25,000 failure rows (0.5% of 5 M) and take a random 5% of the normal rows (about 248,750). The result has about 274,000 rows, of which 9% are failures: enough failure examples, with a known sampling fraction.

Answer: with only 0.5% failures, **stratified sampling by label** is appropriate. It guarantees coverage of the rare class. The sampling fractions must be recorded so that results can be reweighted, and the test data should keep the true 0.5% rate.

6.3 Choosing the number of principal components

Standardise each sensor (z-score), compute the covariance matrix, and rank the eigenvalues. Keep the smallest number of components k whose cumulative explained variance reaches a set threshold, typically 90% to 95%. Confirm with a scree plot (look for the elbow) and, most importantly, with downstream detection performance: keep enough components that failure recall does not drop. The Kaiser rule (keep eigenvalues greater than 1) is a quick check but can keep too many. If, for

example, the first 15 components explain 95% of variance, 200 sensors reduce to 15 features. The number here is illustrative until the sensor data are analysed.

6.4 Part B: discretization of temperature readings

Values (n = 20): 3, 7, 7, 9, 12, 14, 14, 15, 18, 21, 21, 22, 25, 25, 27, 31, 36, 36, 40, 44. Minimum 3, maximum 44.

Equal-width binning, 4 intervals: width = (44 − 3) / 4 = 10.25.

Interval	Values in interval	Count
[3.00, 13.25)	3, 7, 7, 9, 12	5
[13.25, 23.50)	14, 14, 15, 18, 21, 21, 22	7
[23.50, 33.75)	25, 25, 27, 31	4
[33.75, 44.00]	36, 36, 40, 44	4
Total		**20**

Equal-frequency binning, 4 intervals: depth = 20 / 4 = 5 values per bin.

Bin	Values	Range
1	3, 7, 7, 9, 12	3 to 12
2	14, 14, 15, 18, 21	14 to 21
3	21, 22, 25, 25, 27	21 to 27
4	31, 36, 36, 40, 44	31 to 44

The value 21 occurs twice, at the boundary of bins 2 and 3. I put one 21 in each bin so that each bin has five values; the boundary is therefore approximate. Equal-width bins are uneven (5, 7, 4, 4), while equal-frequency bins are even (5, 5, 5, 5) but have unequal widths.

6.5 Part B: supervised discretization by entropy (information gain)

Data: values 10, 12, 15, 18, 22, 25, 28, 31, 35, 40 with classes N, N, N, N, Y, Y, N, Y, Y, Y. There are 5 N and 5 Y, so the entropy before splitting is $H(D) = -(0.5 \log_2 0.5 + 0.5 \log_2 0.5) =$ **1.000**.

Candidate split points are midpoints between adjacent values. For a split point s, $Info(D, s) = (|D_1|/|D|)\, H(D_1) + (|D_2|/|D|)\, H(D_2)$, and Gain = H(D) − Info.

Split point s	Left (≤ s)	Right (> s)	Info(D, s)	Information gain
11	{10} = 1N, 0Y	9: 4N, 5Y	0.892	0.108
13.5	2N, 0Y	8: 3N, 5Y	0.764	0.237
16.5	3N, 0Y	7: 2N, 5Y	0.604	0.396
20	**4N, 0Y**	**6: 1N, 5Y**	**0.390**	**0.610 (best)**
23.5	4N, 1Y	5: 1N, 4Y	0.722	0.278
26.5	4N, 2Y	4: 1N, 3Y	0.876	0.125
29.5	5N, 2Y	3: 0N, 3Y	0.604	0.396
33	5N, 3Y	2: 0N, 2Y	0.764	0.237
37.5	5N, 4Y	1: 0N, 1Y	0.892	0.108

Best split: s = 20 (the midpoint between 18 and 22), with information gain 0.610. The left side is pure (4 N); the right side has 5 Y and 1 N (the 28). The same calculation could be repeated on either side to find a second cut point; that was not required by the question.

6.6 Comparison of unsupervised and supervised discretization

To compare on the same attribute, apply two-interval binning to the labelled values (10 to 40):

Method	Cut point	Left / right split	Information gain	Comment
Equal-width, 2 bins (width 15)	25 (values ≥ 25 go right; effective split between 22 and 25 = 23.5)	4N 1Y | 1N 4Y	0.278	Ignores the class; the cut falls inside the mixed region.
Equal-frequency, 2 bins (depth 5)	Between 18 and 22 (= 20)	4N | 1N 5Y	0.610	Matches the best entropy split here, but only by chance; it would not in general.
Entropy-based (supervised)	20	4N | 1N 5Y	0.610	Chosen to maximise class separation; this is the principled method.

Which suits classification? The supervised method. Unsupervised methods (equal-width and equal-frequency) decide cut points from the values alone, so they may put a boundary where the class does not change, and they can lose the class signal. The entropy-based method places each boundary where the classes change most and is the natural choice for a decision tree. Equal-frequency is a reasonable unsupervised choice when no labels exist, and it can happen to match here.

Q7. Day-4 Design Challenge: Diagnose, Redesign and Validate

Case summary: a bank's loan-default model reported 97% accuracy in testing but missed most defaulters after deployment. Defaults are 3% of loans. Income is highly right-skewed, and occupation is 18% missing.

A. Diagnose the computational bottleneck

Key observation first: a model that predicts "no default" for every loan scores 97% accuracy on this data without learning anything. The reported accuracy therefore says almost nothing. Each defect below is listed with the principle it violates and how it misleads the model or the evaluation.

#	Defect in the pipeline	Principle violated	How it misleads
1	Missing income replaced by the **global mean** computed on the full dataset.	No leakage: fit every statistic on training data only.	Test-set information enters training. The mean is also the wrong fill for a skewed variable: it is pulled up by the tail, and the filled rows create a spike at one value that narrows the distribution.
2	**Occupation dropped** outright (18% missing).	Handle missingness rather than discard information; a missing category can itself be predictive.	Loses a possibly important predictor. The 18% that are missing may be concentrated among defaulters, so dropping the attribute removes their signal.
3	Income **discretised into 5 equal-width bins** with a skewed variable.	Transform skewed data before binning; equal-width bins suit uniform data.	Most applicants fall into the first one or two bins. Resolution is lost exactly where defaults concentrate, and the bins carry little information about the tail.
4	**Min-max normalisation on the full dataset before splitting.**	No leakage: transformations fitted on training data only.	Test-set minimum and maximum influence the scaling of training data. Min-max is also sensitive to a single extreme income, which compresses all other values.
5	**Random 10% of rows** used for training, not stratified.	Preserve the class ratio when sampling; keep the minority class. Reduction should not discard most of the training data.	Discards 90% of training rows, and with 3% defaults the training sample may contain very few defaulters, so the model learns mainly the majority class.
6	Evaluation by **accuracy alone** on an imbalanced test set.	Evaluate with measures suited to imbalanced classes (recall, precision, F1, ROC-AUC, cost).	A trivial majority predictor reaches 97%. Accuracy hides the failure to find defaulters, which is the business cost that matters.

B. Define the design objective

Primary measurable objective: recall on defaulters of at least **0.60** on a held-out, stratified test set (the bank identifies 60% of defaulters).

Secondary objectives:

- **Precision** of at least **0.15**, so that at most about 85% of flagged applicants are not defaulters. This is the trade-off: higher recall means more false alarms, and the bank must accept some.

- **ROC-AUC** of at least **0.70**, to show that the ranking of applicants is useful regardless of the threshold.
- **Accuracy** may fall; this is acceptable and is reported as a secondary measure only.
- **Cost:** assume a missed default costs five times a false alarm. Report total cost as 5 × FN + 1 × FP, and require it to be below the cost of the original model.

Constraints:

- No test data may be used for any fit (imputation, scaling, encoding, feature selection, resampling or threshold choice) before the final evaluation.
- The pipeline must be reproducible (fixed random seeds, saved parameters) and must run on the full data within an acceptable time.
- The model must be explainable to credit officers and regulators (for example logistic-regression coefficients), so complex black-box models require justification.
- Protected or sensitive attributes must not be used directly; fairness across groups should be checked.

C. Design the revised preprocessing strategy

Step	Technique (order)	Alternative considered and why rejected	Fitted on training only?
1	**Stratified train/test split** (70/30) on the default label, before any statistic is computed.	Random split: may leave the test set with very few defaulters, making recall estimates unstable.	The split comes first; everything after is fitted on training data.
2	**Occupation:** keep it; fill missing values with an "Unknown" category and add a missingness indicator (one-hot encoded).	Mode imputation hides that the value was missing. Dropping loses signal (defect 2).	Categories are learned from training data; "Unknown" needs no fit.
3	**Income missing values:** median imputation plus a missing-value indicator.	Global mean: leaks test data and is skewed (defect 1). Class-wise median: uses the label at prediction time, which is not allowed. Regression imputation: more accurate, kept as a sensitivity check.	**Yes**: the median is computed on training rows.
4	**Log transform** of income: log(1 + income).	Box-Cox or Yeo-Johnson: also valid and handle zeros or negatives; log1p is chosen for interpretability. Equal-width binning is rejected (defect 3).	No parameters are learned for log1p.
5	**Standardisation** (z-score) of log-income and age.	Min-max: leakage-sensitive and dominated by extremes (defect 4). Z-score keeps coefficients comparable for logistic regression.	**Yes**: the mean and SD are from training data.
6	**Attribute subset selection:** remove near-zero-variance attributes and one of any pair	PCA: compresses well but gives components that regulators cannot interpret. Dimensionality is modest	**Yes**: correlations and rankings use training data.

	with $\|r\| > 0.9$; rank the rest with a decision-tree importance measure.	here, so selection is enough.	
7	**No row sampling.** Use all training rows.	Random 10% sample (defect 5): discards information. Sampling is used only if computing time forces it, and then it must be stratified.	Not applicable.
8	**Discretisation** (optional, for tree models only): equal-frequency bins for log-income, or entropy-based cut points.	Equal-width: rejected (Q6 showed it ignores class). Logistic regression keeps income continuous, so discretisation is not used by default.	**Yes**: cut points come from training data only.
9	**Cost-sensitive learning:** class_weight = balanced in logistic regression.	SMOTE or random oversampling: creates synthetic or duplicated defaulters and can overfit. Random undersampling: discards majority rows (defect 5). Weighting keeps all data and changes only the loss.	**Yes**: weights come from training counts.
10	**Model and threshold:** logistic regression; threshold set by 5-fold cross-validation on training data to reach recall ≥ 0.60. The test set is used once.	Default threshold 0.5: optimal for accuracy, not for recall. A complex ensemble would be harder to explain (see B).	**Yes**: the threshold uses cross-validated training predictions.

D. Predict before implementing

Written before running the code. These are my predictions; they are to be compared with real results in Section F.

Quantity	**Original pipeline (prediction)**	**Redesigned pipeline (prediction)**
Dispersion of income	Narrowed: a spike at the mean from imputation, and a few bins hold most applicants. Standard deviation understated.	Full spread kept. Log transform reduces skewness from strongly positive towards about zero; coefficient of variation falls substantially.
Class distribution in training data	About 3% defaults in the 10% random sample, but with sampling noise (a few percentage points either way).	Exactly the population rate of about 3% in training, because the split is stratified. Balancing is done through weights, so the counts stay the same.
Accuracy	About 97% (close to the majority-class rate).	Falls, probably to around 80% to 90%, because the model now flags more applicants.
Recall on defaulters	Very low, close to 0 to 10%.	Substantially higher, around 50% to 70%, with precision still low (roughly 10% to 20%) at a 3% base rate.

Core prediction: accuracy will fall and recall will rise. The fall in accuracy is the correct outcome, because the old accuracy was an artefact of predicting the majority class.

E. Implement and validate the redesign

Dataset. UCI German Credit (1000 applicants, 20 attributes, class good/bad, about 30% bad). Two adaptations, both declared: (1) the numeric credit amount, which is right-skewed, plays the role of income; (2) German Credit has no missing values, so missingness is injected at the case rates (10% for the numeric attribute, 18% for the categorical one) with a fixed seed. To match the 3% default rate exactly, use the --subsample-defaults option.

Code. q7_loan_pipeline.py implements both pipelines using pandas and scikit-learn. The original pipeline reproduces defects 1 to 5. The redesigned pipeline implements steps 1 to 10. Run it with:

python q7_loan_pipeline.py --data german_credit.csv --target class --positive bad --subsample-defaults

Metric (same test set for both)	**Original**	**Redesigned**	**Your prediction (D)**
Accuracy	[run]	[run]	80–90% (redesign)
Recall (defaulters)	[run]	[run]	50–70% (redesign)
Precision	[run]	[run]	10–20% (redesign)
F1	[run]	[run]	—
ROC-AUC	[run]	[run]	—
Confusion matrix (TN, FP, FN, TP)	[run]	[run]	—
Income coefficient of variation before / after	[run]	[run]	falls

Smoke test. To check that the script runs, it was executed on a synthetic dataset with the same structure (20,000 rows, about 3% defaults, injected missingness). Those numbers are a code check only and are not evidence about any real dataset. On that data the original pipeline reached accuracy 0.97 with recall 0.00, and the redesigned pipeline reached accuracy 0.64 with recall 0.70, so the direction of the change matched the prediction in D. This is not a substitute for the real comparison.

F. Defend the final design

Comparison with predictions. Once the real results are in, compare each row of the table in E with the prediction in D. My predictions were that accuracy would fall and recall rise, and that income dispersion would narrow after the change to a log transform. The smoke test supports the first two. If the real results differ, the design should be defended on the basis of recall and cost, not accuracy, and the reasons for any difference (for example a different default rate or a different missingness pattern) should be stated.

Defence of the final pipeline.

- Every fit uses training data only, which removes the leakage present in the original pipeline.

- All training rows are kept, and class imbalance is handled by weights rather than by discarding data.
- The model is logistic regression, which is explainable and suited to regulation.
- The decision threshold is set to meet the recall objective, which is the business goal; accuracy is no longer the target.

One remaining risk: concept drift. The coefficients and the recall-targeted threshold were learned on past applicants; if the economy changes, default rates and the relationship between attributes and default will change. The threshold may then no longer meet the recall target. Mitigation: monitor recall and precision monthly on newly labelled loans, re-estimate the threshold on recent data, and retrain on a schedule. A second, smaller risk is that the injected missingness used in the experiment may not match real missingness, which can be non-random.

Appendix: Verification Method and Tool Results

Quantity	**Manual result**	**Python result**	**Match?**
Mean (Q3)	47.625	47.625	Yes
Median (Q3)	44	44.0	Yes
Sample variance / SD (Q3)	222.59 / 14.92	222.59 / 14.92	Yes
Q1 / Q3 (Tukey) (Q3)	37 / 54	37 / 54 (numpy linear: 37.5 / 53.5)	Yes (conventio n)
Mean without 95 (Q3)	45.57	45.565	Yes
Equal-frequency and equal-width bins (Q6)	5,5,5,5 and 5,7,4,4	same	Yes
Information gain at s = 20 (Q6)	0.610	0.6100	Yes
Pearson r (Q5)	0.985	0.9851	Yes
χ^2 (Q5)	33.6	33.60 (Yates: 32.63)	Yes
Min-max, z-score, decimal scaling (Q5)	as tabled	as tabled	Yes
Distance before / after (Q5)	45,000.0 / 0.427	45,000.0 / 0.427	Yes